Newton School of Technology

Advanced Programming

Mid-Semester Examination | Sample Paper with Solutions

This sample shows the **types** of questions in the mid-semester paper and how to answer them on paper. It is shorter than the real paper. Marks show the weight of each question type. Solutions are on the last page.

How to answer

- Multiple choice and Yes/No: write the letter of your answer in the box on the right.
- Short answers: write your answer in the box given. A small box means a number is expected.
- Match the columns: write the letter from Column B in the Answer column.
- Coding: write JavaScript (Node.js, Express) in the box. Write neatly.

Q1. A route handler reads `req.body.title`, but `req.body` is `undefined` even though the client sends JSON. What is most likely missing? **3 M**

(A) `app.use(express.static('public'))`
(B) `app.use(express.json())`
(C) `app.listen(3000)`
(D) `res.json()` at the end of the handler

Q2. A client sends a `POST /books` request with no `title`, but the server requires one. Write the status code the server should return. **3 M**

Answer:

Q3. For the request `GET /books?genre=sci-fi&limit=5`, what does `typeof req.query.limit` evaluate to inside the handler? **3 M**

Answer:

Q4. Yes or No: should a `GET` request change data on the server? **3 M**

(A) Yes **(B)** No

Read the terminal session below and answer Q5 and Q6.

```
$ curl -i -X POST localhost:3000/books \
    -H "Content-Type: application/json" -d '{"title": ""}'
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"error":"title is required"}
```

Q5. Which HTTP method did the client use? **3 M**

Answer:

Q6. Who has to change something for this request to succeed? **3 M**

(A) The client: it must send a non-empty title
(B) The server: it must be restarted
(C) The server: it must add `express.json()`
(D) Nobody: the request already succeeded

Q7. Match each HTTP method in Column A with the action it usually performs on a REST resource in Column B. **3 M**

Column A		Column B		Answer
1	`GET`	P	Removes the resource	
2	`POST`	Q	Updates only the fields that are sent	
3	`PATCH`	R	Reads the resource without changing it	
4	`DELETE`	S	Creates a new resource in a collection	

Q8. Create a book. **8 M**

The app below keeps books in an in-memory array. Write the `POST /books` route.

- The body is JSON, for example `{ "title": "Dune", "year": 1965 }`.
- If `title` is missing, not a string, or empty after trimming, respond `400` with `{ "error": "title is required" }`.
- Otherwise create `{ id, title, year }` with `id = books.length + 1` and the trimmed title, add it to `books`, and respond `201` with the new book.

```
const express = require('express');
const app = express();
app.use(express.json());
const books = [];

// write the POST /books route here

app.listen(3000);
```

Q9. Trace the requests. **8 M**

The server below is running. For each request, write what is printed on the server console (in order) and the status code of the response.

```
const express = require('express');
const app = express();

app.use((req, res, next) => {
  console.log('A', req.method, req.path);
  next();
});

app.get('/books/:id', (req, res, next) => {
  if (req.params.id === '0') return res.status(404).json({ error: 'not found' });
  console.log('B', req.params.id);
  next();
});

app.get('/books/:id', (req, res) => {
  console.log('C');
  res.json({ id: Number(req.params.id) });
});

app.use((req, res) => {
  console.log('D');
  res.status(404).send('no route');
});

app.listen(3000);
```

Request	Console output	Status code
GET /books/7		
GET /books/0		
POST /books/7		